

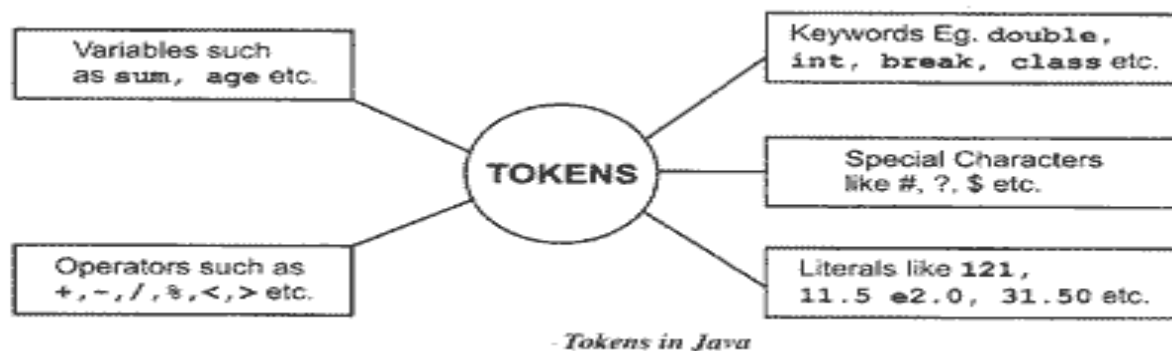
Chapter 2

C++ Tokens

Introduction

A C++ Program is made up of Classes and Methods and in the Methods are the Container of the various statements and a statement is made up of Variables, Constants, operators etc.

Tokens are the various C++ program elements which are identified by the compiler. *A token is the smallest element of a program that is meaningful to the compiler.* Tokens supported in C++ include keywords, variables, constants, special characters, operations etc.



When you compile a program, the compiler scans the text in your source code and extracts individual tokens. While tokenizing the source file, the compiler recognizes and subsequently removes whitespaces (spaces, tabs, newline and form feeds) and the text enclosed within comments. Now let us consider a program

```
//Print Hello
#include<iostream.h>
using namespace std;
int main()
{
    cout<<"Hello C++";
    return(0);
}
```

The source code contains tokens such as using, include, using, namespace, std, ;, int, main, (,), {, cout, <<, "Hello C++", ;, return, (,0,). The resulting tokens are compiled into object code that is capable of being run from your system. Tokens are useful for compiler to detect errors. When tokens are not arranged in a particular sequence, the compiler generates an error message.

C++ language includes five types of tokens and they are:

- Reserved Keywords
- Identifiers
- constant
- Operators
- Separators

2.1 Keywords or Reserved Words

C++ reserved words are keywords that are reserved by C++ for particular uses and cannot be used as identifiers (e.g., variable names, function names, class names). Each of these **62 keywords** has a specific meaning in the C++ programming language. You can't use a keyword for anything other than the meaning described in table 2.1.

A list of 32 Keywords in C++ Language which are also available in C language are given below.

Auto	break	case	Char	Const	continue	default	do
Double	else	enum	Extern	Float	for	goto	if
Int	long	register	Return	Short	signed	sizeof	static
Struct	switch	typedef	Union	unsigned	void	volatile	while

A list of 30 Keywords in C++ Language which are not available in C language are given below.

Asm	dynamic_cast	Namespace	reinterpret_cast	bool
Explicit	new	static_cast	false	catch

Operator	template	Friend	private	class
This	inline	Public	throw	const_cast
Delete	mutable	protected	true	try
Typeid	typename	Using	virtual	wchar_t

2.2 Identifiers

Identifiers are names given to different entries such as variables, structures, and functions. Also, identifier names should have to be unique because these entities are used in the execution of the program.

Identifier naming conventions

- Only alphabetic characters, digits and underscores are permitted.
- First letter must be an alphabet or underscore (_).
- Identifiers are case sensitive.
- Reserved keywords can not be used as an identifier's name.

2.2.1 C++ Variable

A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times.

It is a way to represent memory location through symbol so that it can be easily identified.

Let's see the syntax to declare a variable:

```
type variable_list;
```

The example of declaring variable is given below

```
int x;  
float y;  
char z;
```

Here, x, y, z are variables and int, float, char are data types.

We can also provide values while declaring the variables as given below:

```
int x=5,b=10; //declaring 2 variable of integer type
float f=30.8;
char c='A';
```

Rules for defining variables

- i. A variable can have alphabets, digits and underscore.
- ii. A variable name can start with alphabet and underscore only. It can't start with digit.
- iii. No white space is allowed within variable name.
- iv. A variable name must not be any reserved word or keyword e.g. char, float etc.
- v. Upper and lower case count as different characters.
- vi. A name must not previously be utilized in this block of the program.

Valid variable names:

```
int a;
int _ab;
int a30;
```

Invalid variable names:

```
int 4;
int x y;
int double;
```

Reference Variables

C++ having a new kind of variable known as the reference variable. A reference variable provides an alias.(alternative name) for a previously defined variable. For example, if we make the variable *sum* a reference to the variable *total*, then *sum* and *total* can be used interchangeably to represent the variable.

A reference variable is created as follows:

Syntax:

Data type & <reference name> = <variable name>;

Example:

```
float total=1500;  
float &sum=total;
```

Here sum is the alternative name for variables total, both the variables refer to the same data object in the memory.

A reference variable must be initialized at the time of declaration.

Note that C++ assigns additional meaning to the symbol & here & is not an address operator. The notation float & means reference to float.

Example:

```
int n[10];  
int &x = n[10];  
char &a='n';
```

2.2.2 C++ Data Types

All variables use data-type during declaration to restrict the type of data to be stored. Therefore, we can say that data types are used to tell the variables the type of data it can store. Whenever a variable is defined in C++, the compiler allocates some memory for that variable based on the data-type with which it is declared. Every data type requires a different amount of memory. There are 4 types of data types in C++ language.

Types	Data Types
Basic or Primitive or in-build Data Type	int, char, float, double, etc
Derived Data Type	array, pointer, etc
User Defined Data Type	Structure, union, class, enumeration

1. **Primitive Data Types:** These data types are built-in or predefined data types and can be used directly by the user to declare variables. example: int, char , float, bool etc. Primitive data types available in C++ are:
 - Integer
 - Character
 - Boolean

- Floating Point
- Double Floating Point
- Valueless or Void
- Wide Character

Several of the basic types can be modified using one or more of these type modifiers –

- signed
- unsigned
- short
- long

The following table shows the variable type, how much memory it takes to store the value in memory, and what is maximum and minimum value which can be stored in such type of variables.

Type	Typical Bit Width	Typical Range
char	1byte	-128 to 127 or 0 to 255
unsigned char	1byte	0 to 255
signed char	1byte	-128 to 127
int	4bytes	-2147483648 to 2147483647
unsigned int	4bytes	0 to 4294967295
signed int	4bytes	-2147483648 to 2147483647
short int	2bytes	-32768 to 32767
unsigned short int	2bytes	0 to 65,535
signed short int	2bytes	-32768 to 32767
long int	8bytes	-2,147,483,648 to 2,147,483,647
signed long int	8bytes	same as long int
unsigned long int	8bytes	0 to 4,294,967,295
long long int	8bytes	$-(2^{63})$ to $(2^{63})-1$

unsigned long long int	8bytes	0 to 18,446,744,073,709,551,615
float	4bytes	
double	8bytes	
long double	12bytes	
wchar_t	2 or 4 bytes	1 wide character

The size of variables might be different from those shown in the above table, depending on the compiler and the computer you are using.

Following is the example, which will produce correct size of various data types on your computer.

```
#include <iostream>
using namespace std;

int main() {
    cout << "Size of char : " << sizeof(char) << endl;
    cout << "Size of int : " << sizeof(int) << endl;
    cout << "Size of short int : " << sizeof(short int) << endl;
    cout << "Size of long int : " << sizeof(long int) << endl;
    cout << "Size of float : " << sizeof(float) << endl;
    cout << "Size of double : " << sizeof(double) << endl;
    cout << "Size of wchar_t : " << sizeof(wchar_t) << endl;

    return 0;
}
```

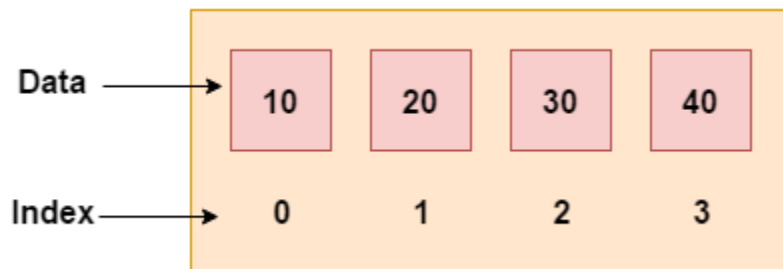
When the above code is compiled and executed, it produces the following result which can vary from machine to machine –

Size of char : 1
Size of int : 4
Size of short int : 2
Size of long int : 4
Size of float : 4
Size of double : 8
Size of wchar_t : 4

2. Derived Data Types

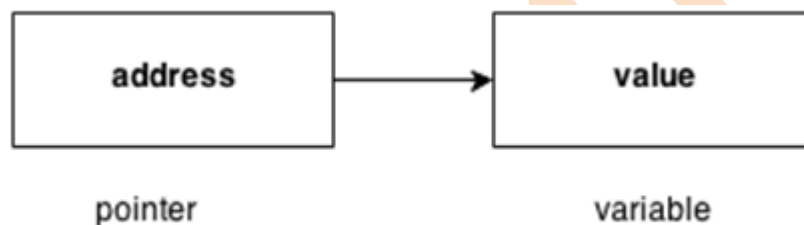
Like other programming languages, array in C++ is a group of similar types of elements that have contiguous memory location.

In C++ **std::array** is a container that encapsulates fixed size arrays. In C++, array index starts from 0. We can store only fixed set of elements in C++ array.



C++ Pointers

The pointer in C++ language is a variable, it is also known as locator or indicator that points to an address of a value.



Advantage of pointer

- i) Pointer reduces the code and improves the performance, it is used to retrieving strings, trees etc. and used with arrays, structures and functions.
- ii) We can return multiple values from function using pointer.
- iii) It makes you able to access any memory location in the computer's memory.

Usage of pointer

There are many usage of pointers in C++ language.

i) Dynamic memory allocation

In c language, we can dynamically allocate memory using malloc() and calloc() functions where pointer is used.

ii) Arrays, Functions and Structures

Pointers in c language are widely used in arrays, functions and structures. It reduces the code and improves the performance.

Symbols used in pointer

Symbol	Name	Description
& (ampersand sign)	Address operator	Determine the address of a variable.
* (asterisk sign)	Indirection operator	Access the value of an address.

The pointer in C++ language can be declared using * (asterisk symbol).

```
int * a; //pointer to int
char * c; //pointer to char
```

Let's see the simple example of using pointers printing the address and value.

```
#include <iostream>
using namespace std;
int main()
{
    int number=30;
    int * p;
    p=&number;//stores the address of number variable
    cout<<"Address of number variable is:"<<&number<<endl;
    cout<<"Address of p variable is:"<<p<<endl;
    cout<<"Value of p variable is:"<<*p<<endl;
    return 0;
}
```

Output:

```
Address of number variable is:0x7ffccc8724c4
Address of p variable is:0x7ffccc8724c4
Value of p variable is:30
```

3. User Defined Data Types

We have used user defined data types such as **struct** and **union** in C. While these data types are legal in C++, some more features have been added to make them suitable for object-oriented programming. C++ also permits us to define another user defined data type known as **class** which can be used, just like any other basic data type, to declare variable. The class variables are

known as objects, which are the central focus of object-oriented programming. More about these data types is discussed later in chapter 5 & chapter 6.

Enumeration

Enumerated type (enumeration) is a user-defined data type which can be assigned some limited values. These values are defined by the programmer at the time of declaring the enumerated type.

When we assign a float value in a character value then compiler generates an error in the same way if we try to assign any other value to the enumerated data types the compiler generates an error. Enumerator types of values are also known as enumerators. It is also assigned by zero the same as the array. It can also be used with switch statements.

For example: If a gender variable is created with value male or female. If any other value is assigned other than male or female then it is not appropriate. In this situation, one can declare the enumerated type in which only male and female values are assigned.

Syntax:

enum enumerated-type-name{value1, value2, value3.....valueN};

enum keyword is used to declare enumerated types after that enumerated type name was written then under curly brackets possible values are defined. After defining Enumerated type variables are created. It can be created in two types:-

1. It can be declared during declaring enumerated types, just add the name of the variable before the semicolon. or,
2. Beside this, we can create enumerated type variables as same as the normal variables.

enumerated-type-name variable-name = value;

By default, the starting code value of the first element of enum is 0 (as in the case of array) . But it can be changed explicitly.

For example: ***enum enumerated-type-name{value 1=1, value2, value3};***

And, The consecutive values of the enum will have the next set of code value(s).

Example

```
#include <iostream.h>
using namespace std;

int main()
{
    // Defining enum Gender
    enum Gender { Male, Female };

    // Creating Gender type variable
    Gender gender = Male;
```

```
switch (gender)
{
case Male:
    cout << "Gender is Male";
    break;
case Female:
    cout << "Gender is Female";
    break;
default:
    cout << "Value can be Male or Female";
}
return 0;
}
```

Output:

Gender is Male

Example 2:

```
#include <bits/stdc++.h>
using namespace std;
```

```
// Defining enum Year
enum year {
```

```
    Jan,
    Feb,
    Mar,
    Apr,
    May,
    Jun,
    Jul,
    Aug,
    Sep,
    Oct,
    Nov,
    Dec
```

```
};
```

```
// Driver Code
```

```
int main()
```

```
{
```

```
    int i;
```

```
    // Traversing the year enum
```

```
    for (i = Jan; i <= Dec; i++)  
        cout << i << " ";  
  
    return 0;  
}
```

Output:

0 1 2 3 4 5 6 7 8 9 10 11

typedef

The **typedef** is a keyword used in C+ programming to provide some meaningful names to the already existing variable in the C program. It behaves similarly as we define the alias for the commands. In short, we can say that this keyword is used to redefine the name of an already existing variable.

Syntax of typedef

1. **typedef** <existing_name> <alias_name>

In the above syntax, '**existing_name**' is the name of an already existing variable while '**alias name**' is another name given to the existing variable.

For example, suppose we want to create a variable of type **unsigned int**, then it becomes a tedious task if we want to declare multiple variables of this type. To overcome the problem, we use a **typedef** keyword.

```
typedef unsigned int unit;
```

In the above statements, we have declared the **unit** variable of type unsigned int by using a **typedef** keyword.

Now, we can create the variables of type **unsigned int** by writing the following statement:

```
unit a, b;
```

instead of writing:

```
unsigned int a, b;
```

Till now, we have observed that the **typedef** keyword provides a nice shortcut by providing an alternative name for an already existing variable. This keyword is useful when we are dealing with the long data type especially, structure declarations.

Let's understand through a simple example.

```
#include <stdio.h>
```

```
int main()
{
typedef unsigned int unit;
unit i,j;
i=10;
j=20;
printf("Value of i is :%d",i);
printf("\nValue of j is :%d",j);
return 0;
}
```

Output

Value of i is :10

Value of j is :20

Using typedef with structures

Consider the below structure declaration:

```
struct student
{
char name[20];
int age;
};
struct student s1;
```

In the above structure declaration, we have created the variable of **student** type by writing the following statement:

```
struct student s1;
```

The above statement shows the creation of a variable, i.e., s1, but the statement is quite big. To avoid such a big statement, we use the **typedef** keyword to create the variable of type **student**.

```
struct student
{
char name[20];
int age;
};
typedef struct student stud;
stud s1, s2;
```

In the above statement, we have declared the variable **stud** of type struct student. Now, we can use the **stud** variable in a program to create the variables of type **struct student**.

2.3. Constants

Constants refer to as fixed values, unlike variables whose value can be altered, constants - as the name implies does not change, they remain constant. Constant must have to be initialized at the time of creating it, and new values cannot be assigned later to it.

- Constants are also called literals.
- Constants can be any of the data types.
- It is considered best practice to define constants using only upper-case names.

There are two other different ways to define constants in C++. These are:

- By using `const` keyword
- By using `#define` preprocessor

Constant Definition by Using `const` Keyword

Syntax:

```
const type constant_name;
```

Example

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
{
    const int SIDE = 50;
    int area;
    area = SIDE*SIDE;
    cout<<"The area of the square with side: " << SIDE <<" is: " << area << endl;
    system("PAUSE");
    return 0;
}
```

It is also possible to put `const` either before or after the type.

```
int const SIDE = 50;
```

or

```
const int SIDE = 50;
```

Constant Definition by Using #define Preprocessor

Syntax:

```
#define constant_name;
```

Example:

```
#include <iostream>
using namespace std;
```

```
#define VAL1 20
#define VAL2 6
#define Newline '\n'
```

```
int main()
{
    int tot;
    tot = VAL1 * VAL2;
    cout << tot;
    cout << Newline;
}
```

2.4 C++ Operators

C++ operator is a symbol that is used to perform mathematical or logical manipulations. C++ language is rich with built-in operators.

In the example below, we use the + **operator** to add together two values:

```
int x = 100 + 50;
```

Although the + operator is often used to add together two values, like in the example above, it can also be used to add together a variable and a value, or a variable and another variable:

```
int sum1 = 100 + 50;    // 150 (100 + 50)
int sum2 = sum1 + 250;  // 400 (150 + 250)
int sum3 = sum2 + sum2; // 800 (400 + 400)
```

C++ divides the operators into the following groups:

- Arithmetic operators

- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators

Arithmetic Operators

Arithmetic operators are used to perform common mathematical operations.

Arithmetic operators are used to perform common mathematical operations.

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value of a variable by 1	$++x$
--	Decrement	Decreases the value of a variable by 1	$--x$

Assignment Operators

Assignment operators are used to assign values to variables.

```
int x = 10;
```

In the example below, we use the **assignment** operator (=) to assign the value **10** to a variable called **x**:

```
int x = 10;
x += 5;
```

A list of all assignment operators:

Operator	Example	Same As
=	$x = 5$	$x = 5$

+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
&=	x &= 3	x = x & 3
=	x = 3	x = x 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3

C++ Comparison Operators

Comparison operators are used to compare two values.

Note: The return value of a comparison is either true (1) or false (0).

In the following example, we use the **greater than** operator (>) to find out if 5 is greater than 3:

```
int x = 5;
int y = 3;
cout << (x > y); // returns 1 (true) because 5 is greater than 3
```

A list of all comparison operators:

Operator	Name	Example
==	Equal to	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Logical Operators

Logical operators are used to determine the logic between variables or values:

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	<code>x < 5 && x < 10</code>
	Logical or	Returns true if one of the statements is true	<code>x < 5 x < 4</code>
!	Logical not	Reverse the result, returns false if the result is true	<code>!(x < 5 && x < 10)</code>

Conditional operator

The conditional operator ?: is called ternary operator as it requires three operands. The format of the conditional operator is:

Conditional_expression ? expression1 : expression2;

If the value of conditional expression is true then the expression1 is evaluated, otherwise expression2 is evaluated.

```
int a = 5, b = 6;  
big = (a > b) ? a : b;
```

The condition evaluates to false, therefore big gets the value from b and it becomes 6.

the sizeof operator

As we know that different types of Variables, constant, etc. require different amounts of memory to store them The sizeof operator can be used to find how many bytes are required for an object to store in memory. For example

```
sizeof (char) returns 1  
sizeof (float) returns 4
```

2.5 Precedence of C++ Operators:

Operator precedence determines the grouping of terms in an expression. This affects how an expression is evaluated. Certain operators have higher precedence than others; for example, the multiplication operator has higher precedence than the addition operator:

For example, $x = 7 + 3 * 2$; here x is assigned 13, not 20 because operator $*$ has higher precedence than $+$, so it first gets multiplied with $3*2$ and then adds into 7.

Here, operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

Category	Operator	Associativity
Postfix	() [] . (dot operator)	Left to right
Unary	++ -- ! ~	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	>> >>> <<	Left to right
Relational	> >= < <=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	

2.6 Separator:

Separators are symbols. It shows the separated code. They describe function of our code.

Name	Use
()	Parameter in method definition, containing statements for conditions,etc.
{}	It is used for define a code for method and classes
[]	It is used for declaration of array
;	It is used to show the separate statement
,	It is used to show the separation in identifier in variable declarartion
.	It is used to show the separate package name from subpackages and classes, separate variable and method from reference variable.

2.7 Promotion and Type Conversion

If you have previous programming experience, then you already know that it is fairly common to assign a value of one type to a variable of another type. If the two types are compatible, then C++ will perform the conversion automatically.

Implicit Type Conversion

The compiler performs type conversion of data items when an expression consists of data items of different types. This is called implicit or automatic type conversion. The rules followed by the compiler for implicit type conversion s shown in table:

Operand1	Operand2	Result
char	int	int
int	long	long
int	float	float
int	double	double
int	unsigned	unsigned
long	double	double
double	float	double

Automatic type conversion rule table

Consider the following statements to illustrate the automatic type conversion

```
float f = 9.0;  
int i = 0;  
i = f / 3;
```

In this expression, the constant 3 will be converted to a float and then the floating point will take place, resulting in 3.33333. This (integer to float) type of conversion, where the variable of a lower size data type (which can hold lower range of values or has lower precision) is converted

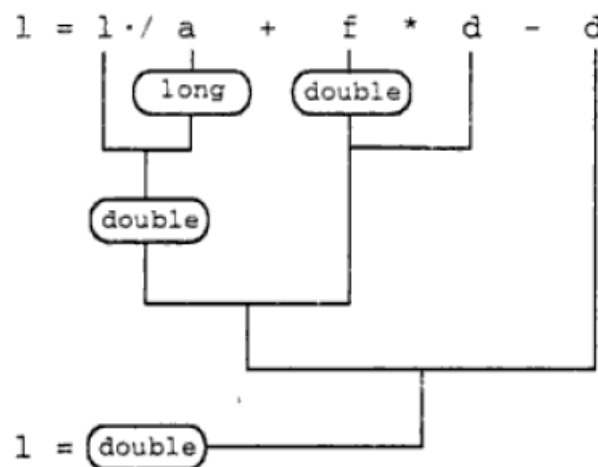
to higher type (which can hold higher range of value or has higher precision) is called *promotion*. But the *lvalue* is an integer variable, hence the result of $f/3$ will be automatically truncated to 3 and the fractional part will be lost. This (float to integer) type conversion, where the variable of higher type is converted to a lower type is called *demotion*.

The implicit conversions thus occurring are also called silent conversions since the programmer is not aware of these conversions. The flexibility of the C++ language, to allow mixed type conversions implicitly, save a lot of effort on the part of the programmer, but at times, it can give rise to bugs in the program.

```
int a, c;  
long l;  
float f;  
double d;  
l = l / a + f * d - d;
```

The variable *a* and *f* are type converted to *long* and *double* respectively. The process of type conversion leading to data promotion or demotion while assigning the computed result (if necessary) as shown in figure:

```
int a;  
long l  
float f  
double d;
```



Automatic Types Conversion

Explicit Type Conversion

Implicit type conversions, as allowed by the C++ language, can lead to error creeping into the program, if adequate care is not taken. Therefore, the use of explicit type conversion is

recommended in mixed mode expressions. It is achieved by typecasting a value of a particular type, into the desired type as follows:

```
(type) expression  
(type) variable_name
```

The expression/variable is converted to the given type. Consider the expression:

```
(float) i+ f;
```

It type cast the variable i of type integer to float. Another syntax for type conversion, which is specific to C++ is as follows:

```
(type) expression  
(type) variable_name
```

Typecasting can also be used to convert from higher type to lower type. For example, if f is a float whose value is 2.7, the expression

```
int(f)
```

evaluates to 2. The following program illustrate the different ways of achieving type conversion

```
#include<iostream.h>  
void main()  
{  
    int i,j;  
    float f;  
    i =12;  
    j = 5;  
    cout<< "when i = " << i << " ;j = " << j << endl;  
    f = i/j;  
    cout<< "I / j = " << f << endl;  
    f = (float) I / j;  
    cout<< "(float) I / j = " << f << endl;  
    f = I / float(j);  
    cout<< "f = I / float(j) = " << f << endl;  
}
```

Output

```
when i = 12 j = 5  
i/j = 2  
(float)i/j = 2.4  
float(i)/j = 2.4  
i/float(j) = 2.4
```